ESG 报告智能解析与指标抽取系统技术手册

本文档记录 ESG 报告智能解析与指标抽取系统的主要实现方法。文档按照系统实际运行顺序展开，依次说明报告采集、PDF 解析、指标检索、大模型抽取、结果校验、数据存储、前端展示和服务器部署等内容。每一部分尽量说明输入是什么、程序做了什么、输出结果保存在哪里，便于后续检查、复现和维护。



目录

1 系统整体处理流程 **5**
1.1 处理逻辑介绍 5
1.2 主要技术环节 6

2 服务器部署技术详解 **7**
2.1 部署结构介绍 7
2.2 Docker Compose 服务配置 8
2.3 持久化目录配置 8
2.4 Dockerfile 构建流程 8
2.5 MinerU 运行环境配置 9
2.6 服务器访问与验证 9

3 运行环境配置 **10**
3.1 后端运行环境 10
3.2 前端运行环境 10
3.3 数据库与部署环境 11

4 数据采集技术详解 **11**
4.1 巨潮资讯网采集逻辑介绍 11
4.1.1 采集目标与数据来源 12
4.1.2 检索请求构造 12
4.1.3 分页遍历与公告去重 13
4.1.4 PDF 下载与安全命名 13
4.1.5 限速控制与异常处理 14
4.1.6 采集输出 14
4.2 PDF 预处理 15
4.3 报告有效性判断 16

5 报告解析技术详解 **16**
5.1 PDF 页面扫描流程 16
5.2 重点页面筛选方法 17
5.3 重点页子 PDF 生成流程 18
5.4 MinerU 解析流程 18
5.5 灰区页面复判方法 19

6 指标体系配置技术详解 19
6.1 ESG 指标字段配置 19
6.2 字段类型与结构型指标 20
6.3 Schema 配置文件作用 20

7 字段证据检索技术详解 21
7.1 文本块来源说明 21
7.2 PyMuPDF 视觉顺序重排 21
7.3 文本块切分方法 22
7.4 BM25 算法关键词召回 22
7.5 Embedding 向量召回 22
7.6 本地向量备用方案 22
7.7 RRF 融合排序 23

8 大模型结构化抽取技术详解 23
8.1 字段抽取流程 23
8.2 Prompt 约束规则 24
8.3 目标年份控制与表格对齐 25
8.4 调用预算控制 26
8.5 输出文件说明 26

9 结果校验与人工复核技术详解 26
9.1 结果校验规则 26
9.2 弱证据降置信方法 27
9.3 置信度校准方法 27
9.4 复核优先级计算 28
9.5 人工复核状态 28

10 数据存储与任务管理技术详解 28
10.1 PostgreSQL 主存储 28
10.2 核心数据表说明 29
10.3 分析相关字段 29

11 后端接口技术详解 30
11.1 任务接口 30
11.2 结果与复核接口 30
11.3 指标分析接口 31

12 前端页面与图表技术详解 **31**
12.1 前端页面结构 31
12.2 ECharts 指标分析 32

13 测试验证 **33**
13.1 单元测试覆盖 33
13.2 异常输入验证 34
13.3 抽取质量评估 34
13.4 召回模式评估 34

14 总结与注意事项 **35**

1 系统整体处理流程

1.1 处理逻辑介绍

系统的输入是一份 ESG 报告 PDF，输出为字段级结构化结果、证据页码、人工复核状态、CSV 文件和指标图表。在实际处理时，程序不会把整份 PDF 直接输入给大模型，而是先扫描页面内容，找出与 ESG 指标相关的文本块，再把这些文本块作为候选证据交给大模型。这样既可以减少输入内容，也方便后续查看每个结果来自哪一页。

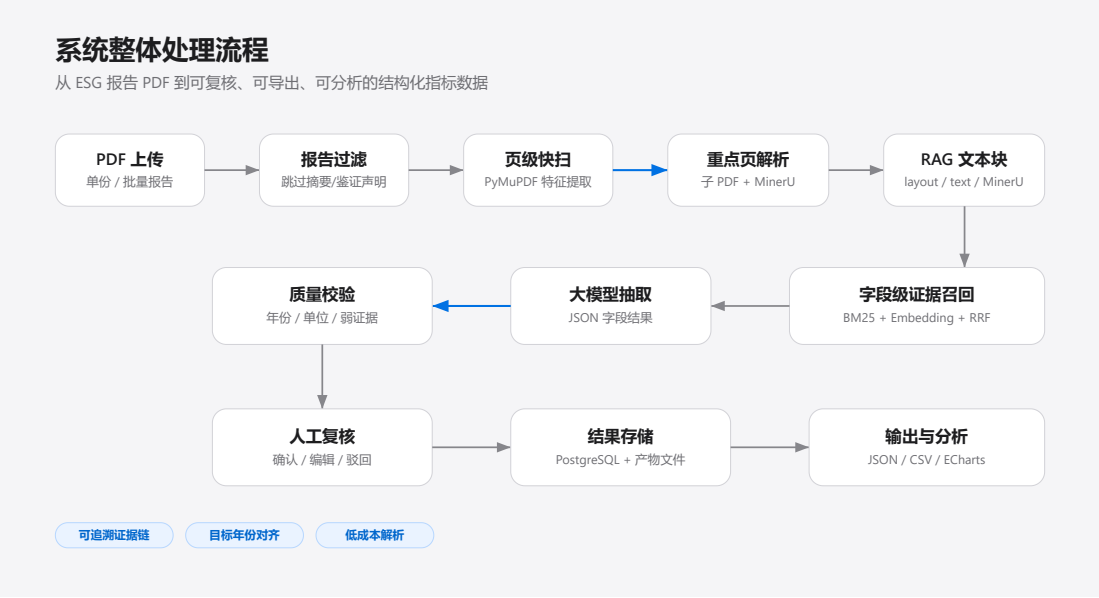


图 1: 系统整体处理流程图

系统主流程如下：

1. 用户在服务器部署后的 Web 系统中上传 ESG 报告 PDF，或通过批量采集脚本准备报告文件。
2. 后端先进行报告有效性判断，过滤鉴证声明、摘要、非完整 ESG 报告等异常输入。
3. 使用 PyMuPDF 对 PDF 做页级快扫，提取文本长度、ESG 关键词、数值密度、表格线索和低文本页特征。
4. 根据页级特征生成 MinerU 解析计划，选择最可能包含指标表或复杂版式的重点页面。
5. 对重点页生成子 PDF，并调用 MinerU 进行深度解析，避免整份报告全量解析。
6. 将 PyMuPDF 文本、PyMuPDF 视觉布局文本和 MinerU 文本统一切分为 RAG 文本块。

7. 针对 60 个 ESG 字段分别检索证据，使用 BM25 算法和 embedding 向量召回，并通过 RRF 融合排序。
8. 将字段定义和候选证据交给 OpenAI-compatible 大模型，输出结构化 JSON 结果。
9. 对抽取结果进行目标年份对齐、单位校验、证据质量评分和复核优先级计算。
10. 前端展示抽取结果和证据，用户可确认、驳回或编辑字段结果。
11. 系统将最终结果保存到 PostgreSQL，并支持 CSV 导出和 ECharts 指标分析。

1.2 主要技术环节

处理环节	使用技术	实现内容
PDF 快扫	PyMuPDF	读取 PDF 文本层，统计每一页的文本长度、关键词、数字密度和表格线索。
重点页解析	MinerU	对表格较多、图文混排明显或文本层较差的页面做深度解析。
子 PDF 生成	PyMuPDF	按照重点页列表从原 PDF 中抽取页面，生成 MinerU 的输入文件。
文本块构建	PyMuPDF layout + PyMuPDF text + MinerU	把不同来源的文本整理成可检索的文本块。
证据召回	BM25 算法 + Embedding + RRF	分别用关键词和语义向量查找字段证据，再进行融合排序。
结构化抽取	OpenAI-compatible LLM	根据字段定义和候选证据生成字段值、单位、年份、证据和置信度。
年份对齐	表格行列规则 + 目标年份策略	在多年份表格中找到目标年份对应的数值，减少跨年误抽。
质量校验	年份、单位、证据质量规则	检查证据是否充分、年份是否一致、单位是否合理。
数据存储	PostgreSQL / SQLite 兼容层	服务器使用 PostgreSQL 保存报告、任务、结果和复核记录。
后端服务	FastAPI + Uvicorn	提供上传、任务查询、结果复核、CSV 导出和指标分析接口。
前端展示	React + TypeScript + Vite	实现报告管理、结果复核、指标对比和导出页面。
可视化	ECharts	绘制定量指标对比图和趋势图。

处理环节	使用技术	实现内容
部署	Docker + Docker Compose	在服务器上启动 API 服务和 PostgreSQL 数据库。

2 服务器部署技术详解

2.1 部署结构介绍

系统以 Docker Compose 方式部署在服务器上。部署后主要包含两个容器：`api` 和 `postgres`。`api` 容器负责运行 FastAPI 后端、React 静态前端和 MinerU 解析环境；`postgres` 容器负责存储报告、任务、抽取结果、复核记录和指标分析数据。

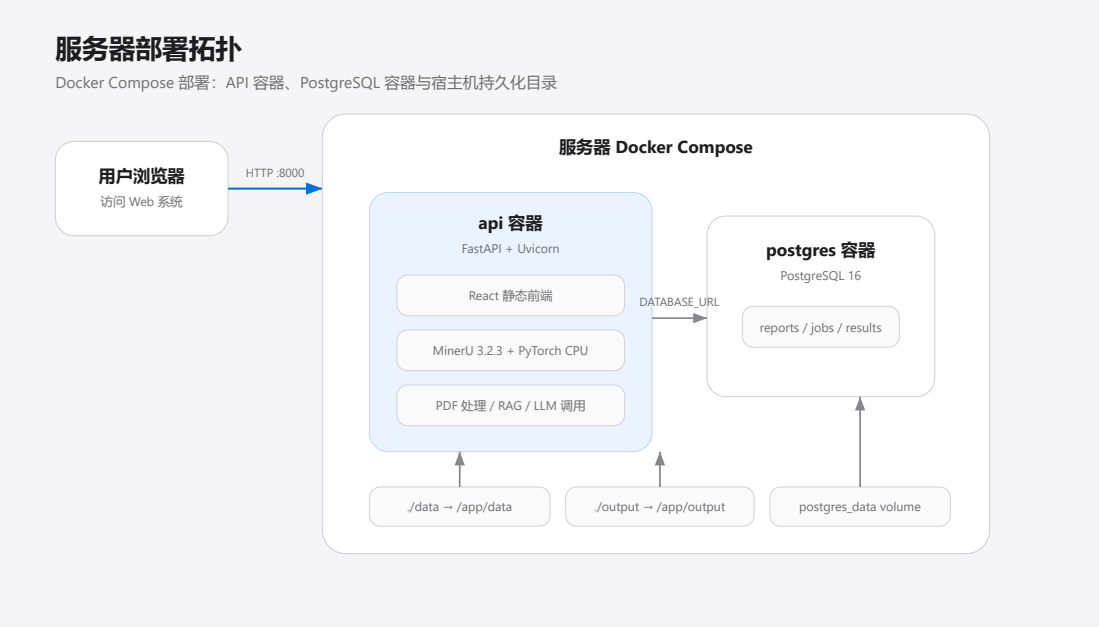


图 2: 服务器部署拓扑图

服务器部署结构如下：

```
用户浏览器
    |
    | HTTP:8000
    v
服务器 Docker Compose
    |-- api 容器
    |      |-- FastAPI 后端
    |      |-- React 静态前端
```

```
    |      |-- MinerU 解析环境
    |      |-- /app/data
    |      `-- /app/output
    `-- postgres 容器
          `-- PostgreSQL 16 数据库
```

2.2 Docker Compose 服务配置

服务	镜像或构建方式	端口	说明
api	项目 Dockerfile 构建	8000:8000	运行 FastAPI、React 静态前端和 MinerU 解析环境。
postgres	postgres :16-alpine	5432:5432	存储报告、任务、抽取结果、复核记录和指标分析数据。

api 容器内部通过如下地址访问 PostgreSQL：

```
postgresql://esg:esg_password@postgres:5432/esg_mineru
```

2.3 持久化目录配置

服务器部署时，系统将关键数据挂载到宿主机目录或 Docker volume，避免容器重建后数据丢失。

宿主机路径或卷	容器路径	作用
./data	/app/data	保存上传 PDF、采集数据和本地备用任务文件。
./output	/app/output	保存 MinerU 缓存、JSON、CSV、评估结果等运行产物。
./configs	/app/configs:ro	只读挂载配置目录和 schema 文件。
postgres_data	/var/lib /postgresql/data	PostgreSQL 数据库持久化存储。

2.4 Dockerfile 构建流程

Dockerfile 使用多阶段构建。

1. 前端构建阶段使用 `node:22-slim`。该阶段先复制前端依赖文件，再执行 `npm ci` 和 `npm run build`，生成 React 静态文件。
2. 后端运行阶段使用 `python:3.11-slim`，复制后端源码、配置文件、`requirements-mineru.txt` 和前端构建产物。
3. 安装系统依赖 `libgl1`、`libglib2.0-0`，满足 OpenCV 和 MinerU 相关依赖。
4. 从 PyTorch CPU index 单独安装 `torch` 和 `torchvision`。
5. 安装 `requirements-mineru.txt` 中固定版本的 MinerU 运行依赖。
6. 安装当前 Python 项目，最后通过 Uvicorn 启动 FastAPI。

容器启动命令如下：

```
uvicorn esg_selective_mineru.api:app --host 0.0.0.0 --port 8000
```

2.5 MinerU 运行环境配置

当前项目新增 `requirements-mineru.txt`，用于固定 MinerU 运行时依赖。该文件包含 `mineru==3.2.3` 以及解析、视觉、OCR、模型下载和文档处理相关依赖。由于 PyTorch 安装源和普通 PyPI 依赖源不同，Dockerfile 将 `torch` 和 `torchvision` 从 PyTorch CPU index 单独安装，再安装 MinerU 依赖清单。

该设计的作用包括：

- 保证服务器容器内具备 MinerU 运行环境。
- 避免每次部署时依赖版本漂移。
- 降低 MinerU 与普通后端依赖混装造成的冲突。
- 让服务器部署时不依赖宿主机已有的 MinerU 环境。

2.6 服务器访问与验证

系统启动后，用户通过服务器 IP 或域名访问 8000 端口。

协议/IP	路径	说明
`http://服务器IP:8000`	`/`	系统首页。
`http://服务器IP:8000`	`/frontend/`	React 静态前端入口。

协议/IP	路径	说明
http://服务器IP:8000	/health	后端健康检查接口。
http://服务器IP:8000	/jobs	任务列表接口。

若服务器前面配置 Nginx 或域名代理，可将外部域名转发到容器的 8000 端口。

3 运行环境配置

3.1 后端运行环境

技术	版本或类型	用途
Python	3.11+	后端主要开发语言。
FastAPI	0.115+	提供报告上传、任务查询、复核保存、CSV 导出和指标分析 API。
Uvicorn	0.30+	ASGI 服务运行器，在容器中监听 8000 端口。
PyMuPDF	1.24+	读取 PDF 文本层、提取 words 坐标、生成重点页子 PDF。
MinerU	3.2.3	对重点页面进行复杂版式和表格解析。
OpenAI SDK	1.40+	调用 OpenAI-compatible 文本模型和 embedding 模型。
psycopg	3.2+	连接 PostgreSQL 数据库。
python-dotenv	1.0+	读取环境变量配置。
python-multipart	0.0.9+	支持 FastAPI 文件上传。
PyTorch CPU Runtime	CPU 版本	为 MinerU 相关模型推理依赖提供运行环境。

3.2 前端运行环境

技术	版本或类型	用途
React	19	构建 Web 交互界面。

技术	版本或类型	用途
TypeScript	5.9	提供类型约束和工程化开发能力。
Vite	8	前端构建工具。
ECharts	6	绘制定量指标横向对比图和趋势图。
CSS	原生 CSS	控制页面布局、表格、按钮、复核面板和图表区域样式。

3.3 数据库与部署环境

技术	用途
PostgreSQL 16	服务器部署主数据库，存储报告、任务、字段结果、复核记录和指标分析数据。
SQLite	开发和离线测试备用数据库。
Docker	构建后端、前端静态文件和 MinerU 运行环境。
Docker Compose	在服务器上一键启动 API 服务和 PostgreSQL 服务。
JSON / CSV	系统中间产物和最终导出格式。

4 数据采集技术详解

4.1 巨潮资讯网采集逻辑介绍

报告采集脚本为 `scripts/collect_a_share_esg_reports.py`，用于从巨潮资讯网（CNINFO）批量采集 A 股上市公司 ESG 相关公告 PDF，并生成报告清单。该脚本不依赖浏览器自动化工具，而是直接调用巨潮资讯公告检索接口，通过关键词、页码、时间范围和股票代码等参数检索公告，再下载公告对应的 PDF 文件。

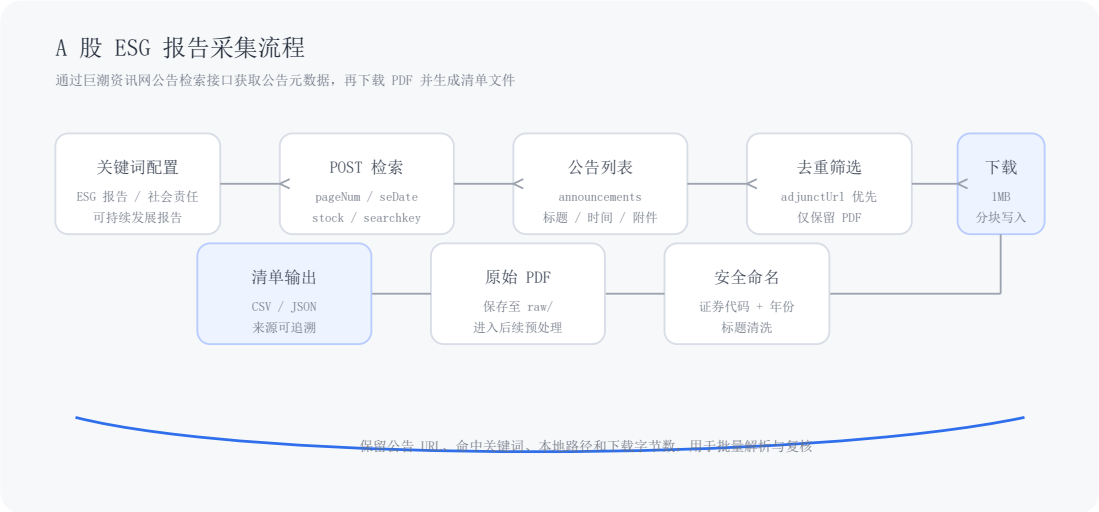


图 3: 报告采集流程图

4.1.1 采集目标与数据来源

爬虫的数据来源为巨潮资讯网公告检索接口：

```
http://www.cninfo.com.cn/new/hisAnnouncement/query
```

公告 PDF 的静态资源根地址为：

```
http://static.cninfo.com.cn/
```

脚本通过公告检索接口获取公告元数据，再根据公告字段中的 `adjunctUrl` 拼接或识别 PDF 下载地址。采集对象主要包括上市公司发布的 ESG 报告、环境、社会和公司治理报告、社会责任报告和可持续发展报告等。

默认关键词包括：

- ESG 报告；
- 环境、社会和公司治理报告；
- 社会责任报告；
- 可持续发展报告。

这些关键词覆盖了 A 股上市公司常见的 ESG 信息披露文件命名方式。用户也可以通过命令行参数重复传入 `--keyword`，扩展或替换默认检索关键词。

4.1.2 检索请求构造

爬虫使用 `urllib.request` 发送 POST 表单请求。检索请求的核心参数包括：

参数	说明
pageNum	当前检索页码。
pageSize	每页返回公告数量。
column	公告所属市场列，脚本中使用 szse。
tabName	检索范围，脚本中使用 fulltext。
searchkey	检索关键词，如 ESG 报告、可持续发展报告等。
stock	股票代码过滤条件，可为空。
seDate	公告时间范围，例如 2021-01-01~2024-12-31。
sortName	排序字段，脚本中按公告时间排序。
sortType	排序方式，脚本中使用倒序。
isHLtitle	是否开启标题高亮。

请求头中设置了 User-Agent、Referer、Content-Type 和 Accept，用于模拟普通网页请求并获取 JSON 格式响应。接口响应中的 announcements 字段即为公告列表。

4.1.3 分页遍历与公告去重

采集函数会按照“关键词—页码”的双层循环遍历公告列表。对于每个关键词，脚本从第 1 页开始请求，直到达到设定页数或当前页无结果为止。为了避免多个关键词命中同一公告，脚本使用 seen 集合进行去重。

去重键优先使用公告 PDF 的 adjunctUrl。如果该字段缺失，则使用证券代码、公告标题和公告时间构造备用键。只有 PDF 文件才会进入下载流程，非 PDF 附件会被跳过。

该策略可以避免以下问题：

- 同一份 ESG 报告同时被多个关键词命中；
- 同一公告在多页结果中重复出现；
- 非 PDF 附件被误下载；
- 重复文件进入后续预处理和抽取流程。

4.1.4 PDF 下载与安全命名

公告下载函数 _download() 通过流式读取方式保存 PDF 文件，每次读取 1 MB 数据块，避免一次性将完整文件加载到内存中。文件保存前会自动创建父目录。

文件命名时，脚本会对证券代码、证券简称和公告标题进行安全清洗。清洗逻辑包括：

- 使用 `html.unescape()` 还原 HTML 实体；
- 去除公告标题中的 `<em>` 高亮标签；
- 将路径非法字符、空白字符替换为下划线；
- 限制标题长度，避免文件名过长。

最终文件名由报告编号和公告标题组成。报告编号包含证券代码、证券简称、公告年份和序号，例如：

```
000001_平安银行_2024_001_2024年度ESG报告.pdf
```

这种命名方式便于后续从文件名中推断公司、年份和报告信息。

4.1.5 限速控制与异常处理

为了减少对目标网站的请求压力，脚本在分页请求和文件下载之间加入 `sleep` 间隔，默认间隔为 0.8 秒。用户可以通过 `--sleep` 参数调整请求间隔。

当 PDF 下载失败时，脚本不会中断整个采集任务，而是打印失败公告的证券代码、标题和错误信息，并继续采集下一条公告。这样可以保证批量采集任务具有一定容错能力。

4.1.6 采集输出

采集完成后，脚本会同时输出 CSV 和 JSON 两种清单文件。CSV 使用 `utf-8-sig` 编码，便于在 Excel 中直接打开中文内容。

产物	说明
`raw/`	保存下载得到的原始 PDF 文件。
`a_share_esg_reports_manifest.csv`	保存报告清单，便于人工查看和后续批处理。
`a_share_esg_reports_manifest.json`	保存 JSON 格式报告清单，便于程序读取。

清单字段包括：

字段	说明
`report_id`	系统生成的报告编号。
`sec_code`	证券代码。

字段	说明
sec_name	证券简称。
org_id	巨潮资讯中的机构编号。
title	公告标题。
announcement_time	公告发布时间。
search_keyword	命中该公告的检索关键词。
pdf_url	PDF 下载地址。
local_pdf	本地 PDF 保存路径。
download_bytes	实际下载字节数。

该 manifest 是后续 PDF 预处理、报告质量检查和批量抽取的重要输入。通过保留原始公告 URL、搜索关键词和本地文件路径，系统可以追溯每份报告的来源，并支持后续重新下载或复核。

4.2 PDF 预处理

预处理脚本为 scripts/preprocess_esg_pdfs.py。该阶段的作用是将原始 PDF 转换为后续流水线可以直接使用的结构化中间产物。预处理不直接完成最终字段抽取，而是为解析计划、RAG 文本块和质量评估提供基础数据。

预处理步骤包括：

1. 读取报告清单。
2. 对 PDF 文件进行统一命名和保存。
3. 使用 PyMuPDF 提取页级文本。
4. 统计每页 ESG 关键词、数值数量、表格线索和低文本页特征。
5. 生成 MinerU 重点页解析计划。
6. 生成 RAG 可用的初始文本块。

产物	说明
page_texts.jsonl	PyMuPDF 页级文本。
page_scan.json	页级关键词、数值密度、表格线索和扫描页识别结果。
parse_plan.json	MinerU 重点页选择计划。
table_candidates.json	表格候选页和表格样本文本行。

产物	说明
pymupdf_chunks.json	RAG 初始文本块。
preprocess_manifest.csv	批量预处理质量概览。

4.3 报告有效性判断

报告过滤逻辑位于 report_filter.py。系统在正式解析前先判断上传文件是否适合进入 ESG 指标抽取流程。该步骤可以避免鉴证声明、审验报告、摘要等非完整报告进入高成本 MinerU 和 LLM 链路。

过滤规则包括：

- 标题层面缺少 ESG、环境社会治理、可持续发展等披露框架信号的传统社会责任报告会被跳过。
- 鉴证声明、审验报告、摘要等支持性文件会被跳过。
- 无法打开或无法解析的 PDF 会记录异常原因。

被跳过的报告不会进入 MinerU 和 LLM 抽取阶段，系统会输出 skip_report.json 和 run_summary.json。

5 报告解析技术详解

5.1 PDF 页面扫描流程

页级快扫由 page_scan.py 完成。该模块使用 PyMuPDF 逐页读取 PDF 文本层，并计算页面特征。页级快扫的目标不是完整理解报告，而是快速判断哪些页面更可能包含 ESG 指标表、绩效表或复杂版式内容。

快扫特征	技术含义	用途
文本长度	当前页可提取文本长度	判断文本层是否完整。
ESG 关键词	环境、社会、治理、碳、员工、安全等关键词命中情况	判断页面主题相关性。
数值数量	数字、百分比、计量单位等出现数量	判断是否可能包含定量指标。
表格线索	指标项、单位、绩效表、连续数值等线索	判断是否可能包含 ESG 数据表。

快扫特征	技术含义	用途
低文本页	文本层较少但页面可能包含图片或扫描内容	标记视觉解析候选页面。
MinerU 分数	多种特征加权后的页面解析价值	判断页面是否进入 MinerU。



图 4: 页级快扫特征示意图

5.2 重点页面筛选方法

解析计划由 `parse_plan.py` 生成。该模块读取 `page_scan.json`，根据页面得分、阈值和最大页数限制，选择需要进入 MinerU 的页面。

解析计划主要包含：

- `page_count`：报告总页数。
- `mineru_pages`：进入 MinerU 的重点页面。
- `visual_fallback_pages`：后续可能需要视觉模型兜底的页面。
- `pages`：每页的解析策略和评分详情。

系统通过 `SELECTIVE_MINERU_MAX_PAGES` 控制每份报告最多进入 MinerU 的页面数，避免对整份报告做高成本解析。

5.3 重点页子 PDF 生成流程

当前版本中，MinerU 调用前新增了重点页子 PDF 生成步骤。该逻辑位于 `mineru_runner.py` 的 `_create_page_subset_pdf()`。系统会根据 `parse_plan.json` 中的 `mineru_pages`，从原 PDF 中抽取这些页面，生成一个只包含重点页的临时 PDF，再交给 MinerU 解析。

该机制的作用包括：

- 减少 MinerU 输入页数，降低解析时间和资源占用。
- 避免 MinerU 对封面、目录、普通正文页等低价值页面重复解析。
- 保留选择性解析策略的一致性，使 `mineru_jobs.json` 中可以记录实际解析页数。
- 便于测试和追踪 MinerU 实际处理了哪些页面。

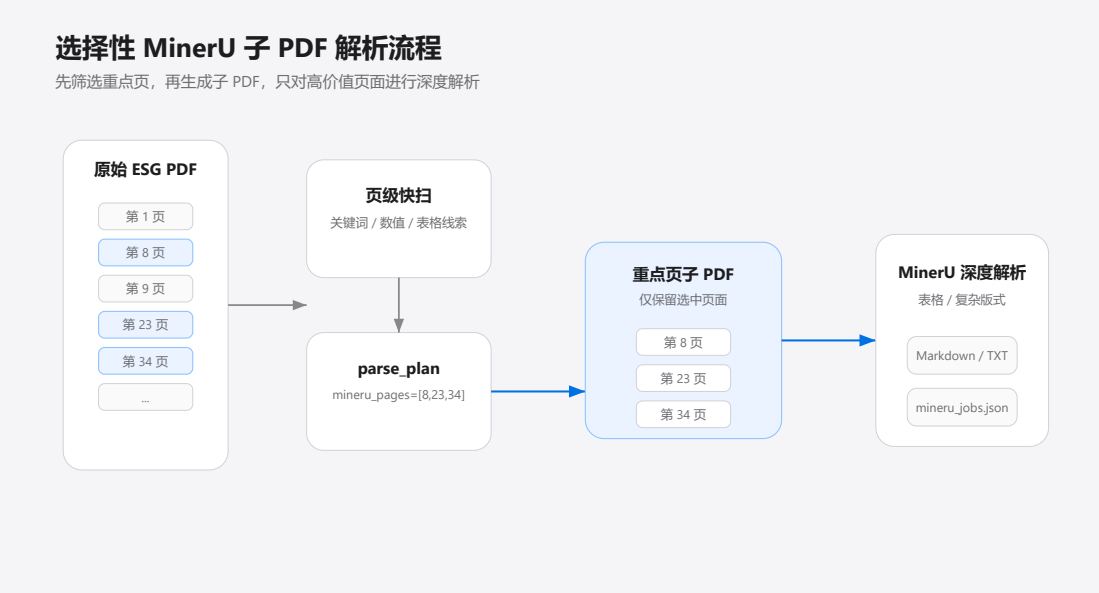


图 5: 选择性 MinerU 子 PDF 解析流程图

5.4 MinerU 解析流程

MinerU 调用逻辑位于 `run_mineru()`。该函数接收 PDF 路径、输出目录、命令模板、超时时间和选中页列表。运行后，系统会记录以下信息：

- 是否尝试调用 MinerU。
- MinerU 返回状态。
- 标准输出和错误输出尾部。

- 实际执行命令。
- 实际输入 PDF 路径。
- 选中的页面列表和页面数量。

这些信息写入 mineru_jobs.json，用于排查解析失败、超时或页面选择异常。

5.5 灰区页面复判方法

灰区页面复判由 mineru_page_judge.py 完成。对于页面分数处于低阈值和高阈值之间的页面，系统可调用 LLM 判断是否值得进入 MinerU。该功能通过 MINERU_LLM_REVIEW_ENABLED 控制是否启用，并通过 MINERU_LLM_REVIEW_MAX_PAGES 限制最大复判页数。

6 指标体系配置技术详解

6.1 ESG 指标字段配置

系统使用 A 股 ESG 60 字段指标体系。字段定义位于 configs/schema/a_share_v5.py，字段加载逻辑位于 schema_loader.py。每个字段不仅用于最终输出，也用于证据召回、Prompt 约束、质量校验和前端展示。

字段属性	说明
field_key	字段唯一编码。
name_cn	字段中文名称。
category	ESG 维度，通常为 E、S、G。
indicator_type	指标类型，如定量、定性、摘要等。
value_type	字段值形态，如单一数值、文本、混合信息或结构型分布。
aliases	字段别名。
search_terms	召回搜索词。
expected_units	期望单位。
required_any	证据中优先要求出现的字段语义词。
forbidden_any	证据中出现后需要降权或排除的冲突语义词。
domain_knowledge	字段相关说明和判定知识。

6.2 字段类型与结构型指标

当前指标体系不仅区分定量、定性和混合型指标，还通过 `value_type` 进一步约束字段值形态。对于员工性别结构、员工年龄结构、员工学历结构、主要污染物排放清单和董事会构成等字段，系统将其配置为结构型字段。结构型字段不应抽取单一 `value`，而应在 `summary` 中保留完整分布，例如男性员工、女性员工及对应人数，或各年龄段、各学历层级的完整人数分布。

该设计用于避免“员工学历结构”被抽成“员工总数 8401 人”这类口径错误。结构型字段在 Prompt 中会被明确提示为分布结构；质量校验阶段也会清空不可靠的单一 `value` 和 `unit`，保留证据和摘要，交由前端复核。

6.3 Schema 配置文件作用

使用位置	作用
证据召回	将字段名、别名、搜索词和单位组合为查询。
大模型抽取	约束模型必须返回哪些字段以及字段类型。
质量校验	判断单位、年份和字段类型是否合理。
前端展示	按 E/S/G 维度和指标类型组织字段。
指标分析	筛选定量指标用于横向对比和趋势分析。



图 6: 60 字段指标体系样例表

7 字段证据检索技术详解

7.1 文本块来源说明

文本块构建逻辑位于 `chunks.py`。当前系统的 RAG chunks 来自三类文本：

1. **PyMuPDF 视觉布局文本**：通过 `page.get_text("words")` 提取词和坐标，再按页面视觉位置重排行列。
2. **PyMuPDF 普通文本**：通过普通文本层提取，适合正文段落。
3. **MinerU 文本**：读取 MinerU 输出的 Markdown 或文本文件，补充复杂表格和版式内容。

7.2 PyMuPDF 视觉顺序重排

当前版本新增 `_words_to_visual_lines()`，用于将 PyMuPDF 提取的 words 按视觉位置排序。该函数先根据 y 坐标聚合为行，再按 x 坐标排列每行内的词，最后拼接为接近人眼阅读顺序的文本。

该机制主要解决 PDF 表格文本提取中的一个常见问题：普通文本提取可能打乱表格行列顺序，导致年份、单位和数值错位。视觉顺序文本可以更好地保留表格横向结构，为后续目标年份对齐提供基础。



图 7: PyMuPDF 视觉布局文本重排示意图

7.3 文本块切分方法

系统将文本切分为适合检索和模型输入的小段，每个 chunk 保留来源信息。典型字段如下：

- `chunk_id`：文本块编号。
- `source`：文本来源，如 `pymupdf_layout`、`pymupdf`、`mineru`。
- `page`：来源页码。
- `text`：文本内容。

7.4 BM25 算法关键词召回

BM25 算法召回由 `SimpleRetriever` 实现。系统会从字段定义中提取字段名、字段编码、主题词、别名、搜索词和期望单位，将其组合为查询文本，再计算每个 chunk 与查询文本的相关性。BM25 算法适合字段名称、单位和关键词直接出现的场景。

7.5 Embedding 向量召回

Embedding 向量召回由 `EmbeddingVectorRetriever` 实现。系统在 hybrid 模式下可调用 OpenAI-compatible Embedding API，将字段查询和文本块转换为向量，再通过 cosine similarity 计算语义相似度。

启用 embedding 的关键配置为：

```
RETRIEVER_MODE=hybrid
RETRIEVER_VECTOR_BACKEND=embedding
EMBEDDING_MODEL=text-embedding-v4
```

Embedding 召回适合处理报告中表达方式不完全一致的情况。例如字段定义为“温室气体排放总量”，报告中可能写作“GHG emissions”“二氧化碳当量排放”或“范围一及范围二排放合计”。语义向量可以提高这类证据的召回概率。

7.6 本地向量备用方案

如果 embedding API key 缺失或调用失败，系统会自动回退到本地字符 n-gram TF-IDF 向量，并在结果中标记 `vector_backend: local_fallback`。本地向量用于异常兜底和离线测试，不作为服务器正式运行时的主要召回能力。

7.7 RRF 融合排序

`HybridRetriever` 会同时执行 BM25 算法和 embedding 向量召回，再使用 RRF 方法融合结果。RRF 可以避免单一召回方式带来的偏差：BM25 算法更重视关键词精确命中，embedding 更重视语义相似度，融合后优先保留两类方法都认为相关的证据。

在 RRF 融合后，系统还会执行字段级重排。字段级重排会根据字段名、别名、搜索词、必需语义词和期望单位计算 `field_match_score`，并对命中 `forbidden_any` 的证据进行降权。该步骤用于解决相近指标之间的串扰，例如“总用水量”不应被“能源消耗总量”或“绿电”表格挤占，“供应商 ESG 或可持续评估”也不应仅因出现“供应商数量 1436 家”就被判定为已披露 ESG 评估数量。

除 BM25 算法和向量召回结果外，系统还会从全部 chunks 中补充字段精确候选池。只要文本块明确包含字段名、别名或搜索词，就会进入最终重排候选。该机制可以避免表格页在向量排名中靠后时被 `RAG_TOP_K` 截断，尤其适用于绩效表中字段密集、多个指标共享同一页的情况。当前默认每个字段最终送入 LLM 的证据块数量为 `RAG_TOP_K=8`。

字段	说明
`retrieval_source`	证据来源于 BM25 算法、vector 或两者。
`bm25_rank`	BM25 算法排名。
`vector_rank`	向量召回排名。
`bm25_score`	BM25 算法分数。
`vector_score`	向量相似度分数。
`hybrid_rank`	融合后排名。
`vector_backend`	向量后端类型。
`field_match_score`	字段级重排分数，用于衡量证据块是否真正命中当前字段语义。

8 大模型结构化抽取技术详解

8.1 字段抽取流程

结构化抽取逻辑位于 `extractor.py`。该模块将 schema、RAG chunks、字段级证据和模型调用串联起来，输出最终的 JSON 和 CSV。

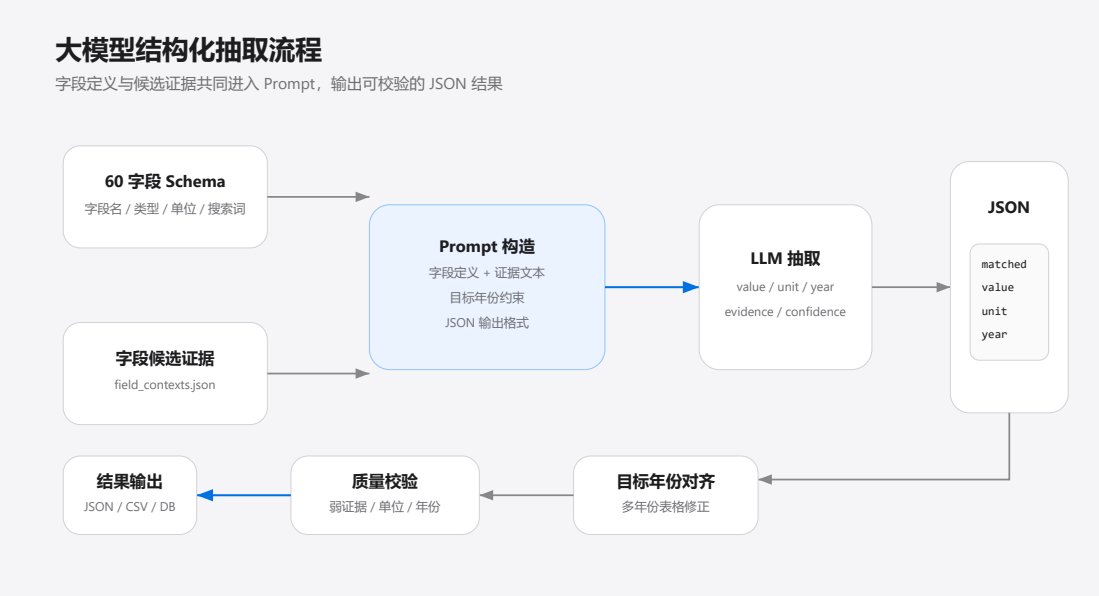


图 8: 大模型结构化抽取流程图

```
加载 60 字段 schema
        |
构建 RAG chunks
        |
按字段召回候选证据
        |
按批次调用 LLM
        |
解析 JSON 返回结果
        |
目标年份表格对齐
        |
质量校验
        |
写出 JSON / CSV
```

8.2 Prompt 约束规则

Prompt 构造逻辑位于 `llm_client.py`。系统将字段定义和候选证据传给模型，并明确要求模型输出固定 JSON 结构。主要约束包括：

- 每个字段必须返回一条结果。
- `matched=false` 表示未披露或证据不足。

- 定量字段优先抽取 value、unit、year。
- 定性字段给出 summary。
- value_type=structure 的字段必须返回完整分布摘要，不应返回单一数值。
- evidence 必须是证据中的短句。
- confidence 取值为 0 到 1。
- 优先抽取目标年份对应的数据。

8.3 目标年份控制与表格对齐

当前版本在 extractor.py 中新增目标年份表格对齐逻辑，核心函数为 _align_result_to_target_year()。该函数用于修正多年份表格中的误抽问题。

ESG 报告中常见如下多年份表格：

指标	2020	2021	2022
年度培训总时长（小时）	35000	36500	38700

如果目标年份是 2022，模型可能因为证据行换行或 PDF 文本顺序问题抽到 36500。系统会在候选证据中重新寻找目标字段所在行和年份表头，并根据目标年份位置取出对应数值，最终将 value 修正为 38700。

该机制支持以下情况：

- 年份横向排列的表格。
- 年份表头被 PDF 提取为多行的情况。
- PDF 文本顺序不完全符合年份顺序的情况。
- 表格中同一证据块包含多个字段和多组数字的情况。
- LLM 返回 matched=false，但候选证据中存在可确定年份和值的字段表格行。

修正后，系统会将 reason 标记为 year_aligned_from_table，便于后续检查。

如果 LLM 因证据片段不完整而返回未抽取，但后处理在候选证据中找到“年份表头 + 字段行 + 单位 + 多年数值”的确定性结构，系统会将结果从 matched=false 修正为 matched=true，并在 reason 中标记 deterministic_year_table_match。例如绩效表中出现“2021 年、2022 年、2023 年”和“总用水量吨 73,546,206 70,439,637 69,113,761”时，目标年份为 2023 的结果会被确定性补为 69,113,761 吨。

8.4 调用预算控制

为了控制模型调用成本，系统提供如下配置：

配置项	说明
LLM_MAX_CALLS_PER_REPORT	每份报告最大模型调用次数。
LLM_FIELD_BATCH_SIZE	每次模型调用处理字段数。
RAG_TOP_K	每个字段召回的证据块数量。
LLM_CONTEXT_CHARS	每个证据块送入模型的最大字符数，默认 1800。

当调用次数达到上限后，系统会为剩余字段生成空结果并记录原因，保证整体流程不中断。

8.5 输出文件说明

文件	说明
rag_chunks.json	RAG 文本块。
field_contexts.json	每个字段召回的候选证据。
extraction_results.json	字段抽取完整结果。
extraction_results.csv	字段抽取表格结果。
extraction_summary.json	抽取摘要。

9 结果校验与人工复核技术详解

9.1 结果校验规则

质量校验逻辑位于 quality.py。该模块对 LLM 返回结果进行二次处理，补充标准化值、单位警告、年份警告、证据评分和质量提示。

校验项	说明
年份校验	判断字段年份是否与目标报告年份一致。
单位校验	判断单位是否缺失或是否与字段期望单位不符。

校验项	说明
证据评分	判断证据文本是否足以支撑字段结果。
字段语义检查	判断证据是否命中字段名、别名或必需语义词，避免只因 evidence 出现在某行就相信该行。
目标年份证据检查	如果定量字段证据中不包含目标年份，则标记 `target_year_not_in_evidence`。
多数字歧义检查	如果证据中存在多个数字但没有目标年份，则标记 `ambiguous_multi_number_evidence_without_target_year`。
弱证据检查	证据评分低于阈值时标记 `weak_evidence`。

9.2 弱证据降置信方法

当前版本对多数字但缺少目标年份的证据进行了更严格处理。若一个定量字段已经匹配成功，但证据中出现多个数字且没有目标年份，系统会认为该结果存在歧义，并将置信度限制到不高于 0.35，同时将证据评分限制到较低区间，必要时将结果重新置为 `matched=false`。

该机制主要用于处理如下情况：

- 多年份表格中只截取了数值行，没有截取年份行。
- 证据中同时出现多个年份或多个口径，但模型只返回一个数值。
- 证据无法明确说明当前值对应目标年份。

9.3 置信度校准方法

LLM 返回的 `confidence` 不直接作为最终置信度。质量校验会根据证据质量重新校准：如果字段名或别名、数值、单位和目标年份关系清楚，且没有严重质量警告，系统会将低估的置信度提升到与证据评分相匹配的水平；如果证据存在年份不明、单位异常、字段语义不匹配或多数字歧义，则会下调置信度。

该机制用于解决“数字正确但置信度只有 35%”的问题。例如证据为“员工总人数：8,401 人”时，虽然证据句本身不重复出现目标年份，但它是单一数值证据，且字段语义、数值和单位都明确，系统会将置信度校准到较高区间；而“员工培训总时长小时 36,500 38,700 39,600”这类多数字证据若没有年份表头，则仍会被视为低置信。

9.4 复核优先级计算

复核优先级计算逻辑位于 `api.py` 的 `_review_priority()`。系统会根据字段风险计算 0 到 100 的优先级。影响因素包括：

- 字段未命中。
- 置信度较低。
- 缺少证据。
- 缺少来源页码。
- 证据质量评分较低。
- 年份异常。
- 单位异常。
- 人工已编辑但仍需确认。

9.5 人工复核状态

状态	含义
`pending`	待复核。
`approved`	已确认。
`rejected`	已驳回。
`edited`	已编辑。

用户可以在前端查看字段值、证据、页码、置信度和质量警告，并对 value、unit、year、evidence 和备注进行修正。

10 数据存储与任务管理技术详解

10.1 PostgreSQL 主存储

服务器部署时，系统通过 `DATABASE_URL` 连接 PostgreSQL，并将其作为主数据库。SQLite 仅作为开发和离线测试备用路径。数据库访问由 `job_store.py` 统一封装，业务层不直接区分 SQLite 与 PostgreSQL。

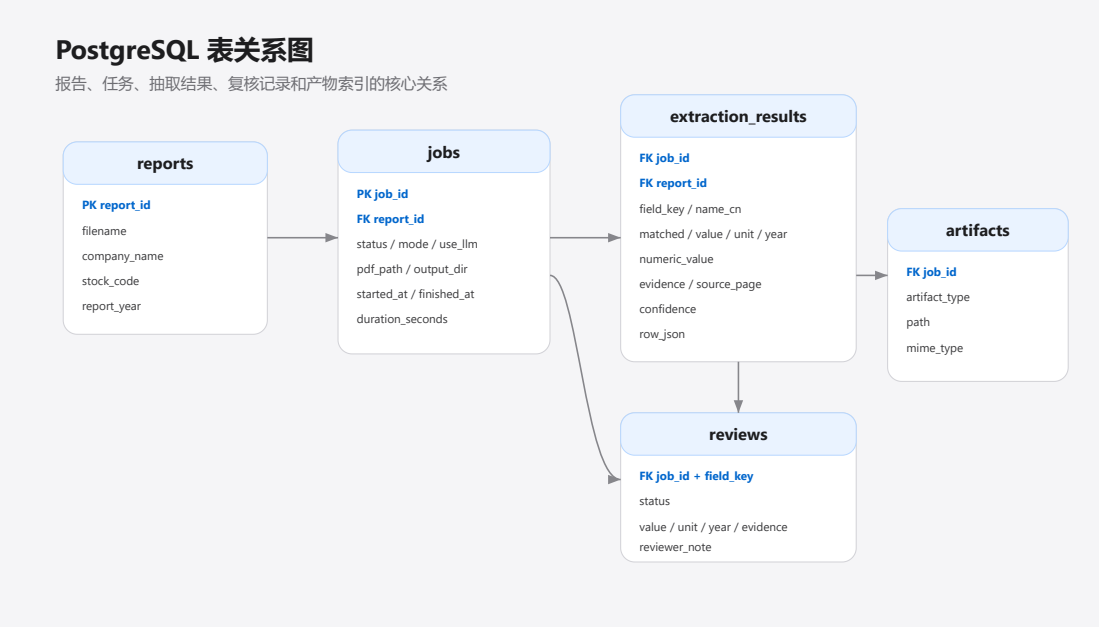


图 9: 数据库表关系图

10.2 核心数据表说明

表	说明
reports	报告元数据，包括文件名、公司名、证券代码和报告年份。
jobs	任务信息，包括状态、模式、耗时、输出目录和错误信息。
reviews	人工复核记录。
extraction_results	字段抽取结果。
artifacts	任务产物索引。

10.3 分析相关字段

字段	所属表	作用
company_name	reports	公司名称，用于横向对比。
stock_code	reports	证券代码。
report_year	reports	报告年份。
started_at	jobs	任务开始时间。
finished_at	jobs	任务结束时间。

字段	所属表	作用
duration_seconds	jobs	任务耗时。
timing_json	jobs	分阶段耗时详情。
report_id	extraction_results	结果关联报告。
numeric_value	extraction_results	用于 ECharts 图表分析的数值字段。

11 后端接口技术详解

11.1 任务接口

方法	路径	功能
POST	/reports	上传单份报告并创建任务。
POST	/reports/batch	批量上传报告并创建任务。
GET	/jobs	查询任务列表。
GET	/jobs/{job_id}	查询任务详情。
POST	/jobs/{job_id}/retry	重跑任务。
DELETE	/jobs/{job_id}	删除任务。

11.2 结果与复核接口

方法	路径	功能
GET	/jobs/{job_id}/summary	查询任务摘要。
GET	/jobs/{job_id}/results	查询字段抽取结果。
GET	/jobs/{job_id}/quality	查询质量统计。
GET	/jobs/{job_id}/reviews	查询复核记录。
PUT	/jobs/{job_id}/reviews/{field_key}	保存字段复核。
GET	/jobs/{job_id}/export.csv	导出复核后 CSV。
GET	/jobs/{job_id}/artifacts	查询任务产物。

11.3 指标分析接口

方法	路径	功能
GET	/metrics/options	获取年份、企业和指标选项。
GET	/metrics/compare	企业横向指标对比。
GET	/metrics/trend	单企业指标趋势分析。

12 前端页面与图表技术详解

12.1 前端页面结构

前端位于 frontend-react/，采用 React + TypeScript + Vite 开发。前端构建后作为静态文件复制到后端镜像中，由 FastAPI 挂载访问。



图 10: React 前端页面结构图

页面	功能
项目总览	展示任务数量、完成数、失败数和待复核数量。
报告管理	上传、查看、重跑、删除报告。
结果复核	查看字段结果、证据、置信度并进行人工修正。
指标对比	对多份报告的同一字段进行横向比较。
指标分析	使用 ECharts 展示定量指标图表。

页面	功能
导出中心	下载复核后的 CSV。

12.2 ECharts 指标分析

系统使用 ECharts 展示定量 ESG 指标。当前可视化方式包括：

- 企业横向对比柱状图：展示同一年份、同一指标下不同企业的指标值。
- 企业趋势折线图：展示同一企业、同一指标在不同年份的变化趋势。
- 非定量指标证据卡片：对无法数值化的定性指标，以证据和摘要方式展示。

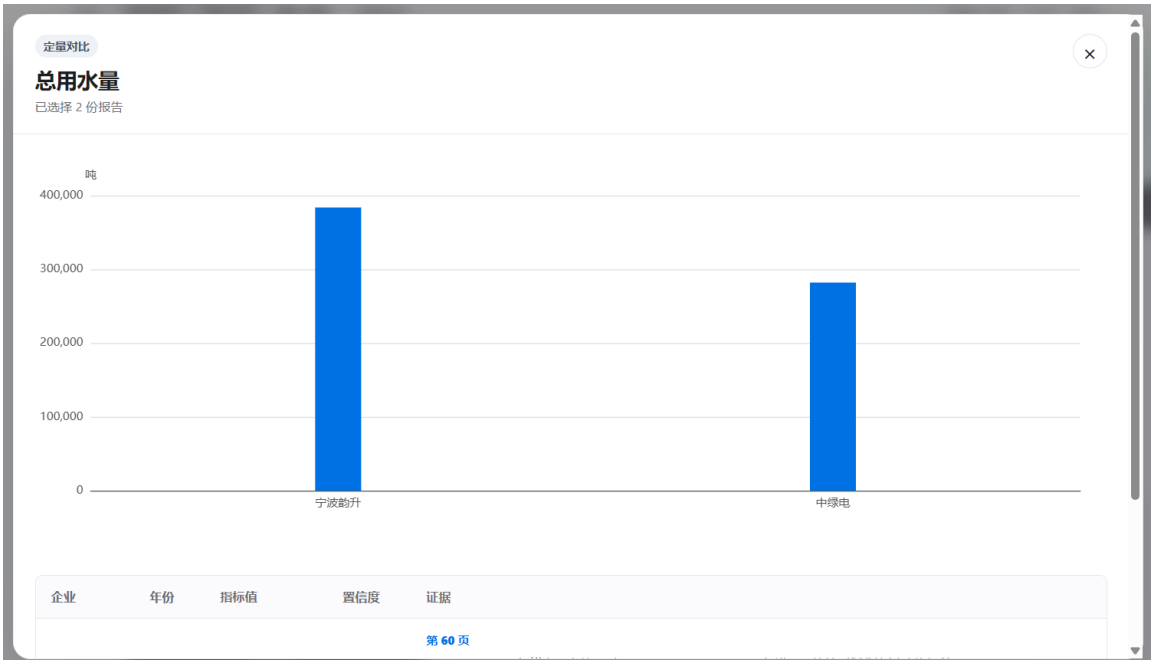


图 11: ECharts 企业横向对比截图

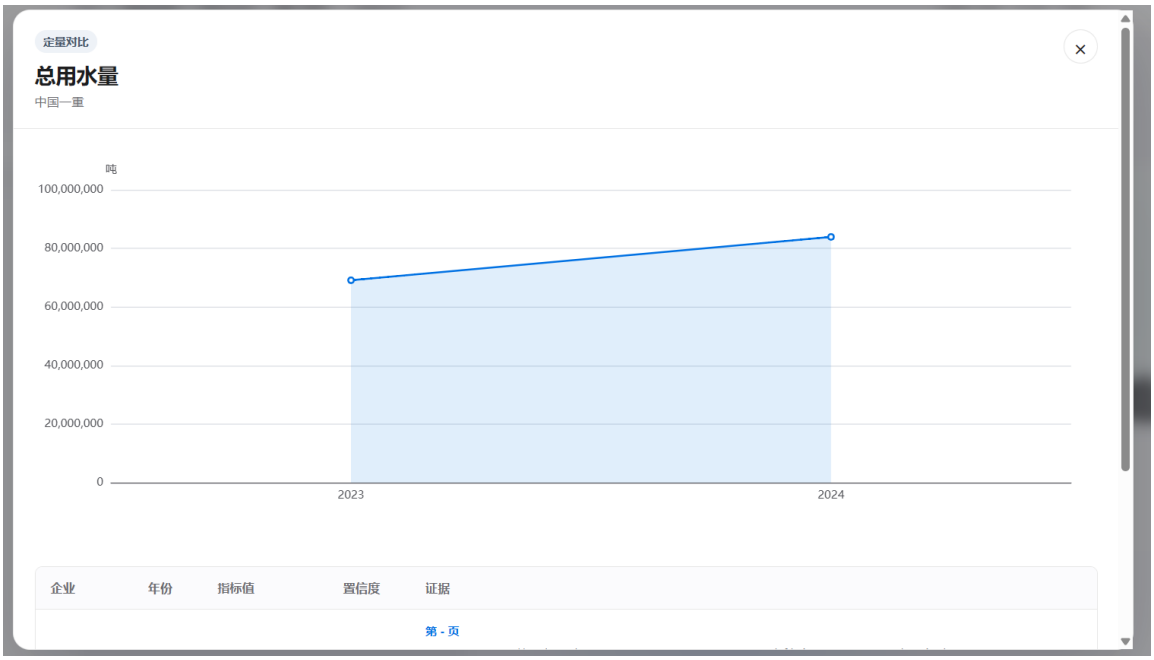


图 12: ECharts 单企业趋势分析截图

13 测试验证

13.1 单元测试覆盖

测试文件	测试内容
tests/test_report_filter.py	报告有效性过滤规则。
tests/test_parse_plan.py	重点页解析计划生成。
tests/test_retriever.py	BM25 算法、hybrid 和 embedding 召回逻辑。
tests/test_mineru_runner.py	重点页子 PDF 生成是否只保留选中页面。
tests/test_chunks.py	PyMuPDF words 是否能按视觉位置重排为表格行。
tests/test_llm_prompt.py	Prompt 中字段类型、证据长度和结构型字段约束。
tests/test_structure_fields.py	结构型字段不会被单一数值污染。
tests/test_year_alignment.py	目标年份表格对齐、未抽取表格救援、年份缺失证据和弱证据降置信。

13.2 异常输入验证

系统对以下异常输入进行处理：

- 非完整 ESG 报告。
- 鉴证声明或审验报告。
- 摘要文件。
- 无法解析的 PDF。
- 重复上传的 PDF。

异常输入不会进入 MinerU 和 LLM 抽取链路，系统会记录跳过原因。

13.3 抽取质量评估

抽取质量评估主要关注：

- 字段命中率。
- value 准确率。
- evidence 可用率。
- Top-K 证据召回成功率。
- 目标年份对齐准确性。
- 结构型字段完整性。
- 低置信结果人工复核命中率。
- 平均处理时间。
- MinerU 页面调用比例。

为了判断系统是否可以稳定用于报告分析，建议构建一个小规模人工评估集。评估集可先选择 5 到 10 份不同行业、不同版式的 A 股 ESG 报告，对 60 个字段进行人工标注或重点字段标注。每份报告至少记录字段是否披露、正确值、单位、年份、证据页和证据文本。系统输出后，再按字段统计命中率、值准确率、证据准确率和人工复核工作量。若定量核心字段的值准确率和证据准确率达到预设阈值，且低置信结果主要集中在报告未披露或证据确实模糊的字段，则可以认为当前架构基本稳定。

13.4 召回模式评估

系统支持比较以下召回模式：

- simple：BM25 算法召回。
- hybrid + embedding：BM25 算法与 embedding 向量召回融合。
- local fallback：embedding 不可用时的本地向量兜底。

当前正式配置使用 `hybrid + embedding`，并叠加字段级重排和字段精确候选池。评估时可人工检查每个字段召回的 Top-K 证据是否真正支持抽取结果，并记录 BM25 算法、embedding、RRF 和字段重排各自带来的改进。若某类字段长期召回失败，再考虑引入向量数据库或专门的表格结构化索引。

14 总结与注意事项

本文档围绕“智析万卷：面向 ESG 报告的自适应布局分析与要素提取”的实际实现流程，说明了系统从报告上传、页面筛选、选择性 MinerU 解析、RAG 文本块构建、字段级证据召回、大模型结构化抽取，到结果校验、人工复核、数据库存储和前端可视化的完整链路。系统通过规则筛选、版式解析、BM25 算法、embedding、RRF 融合排序和字段级重排共同定位证据，并通过目标年份对齐、结构型字段约束和质量校验提高抽取结果的可复核性。

使用系统时需要注意三点。第一，字段结果应始终和证据文本、来源页码一起查看，不能只看抽取值本身；第二，低置信度、年份异常、单位异常和证据不足的结果应优先人工复核；第三，报告版式、字段命名和披露口径在不同企业之间差异较大，因此单份报告的运行结果不能代表整体效果，正式使用时应结合多份报告的复核记录判断系统表现。系统保留 JSON、CSV、数据库记录和前端复核状态，便于后续追溯每个字段的来源和处理过程。